CARAT: Class-Aware Robust Aggregation for Model-Poisoning-Resilient Federated Learning

Anonymous Author(s)
Affiliation
Address
email

Abstract

Federated learning is vulnerable to model-poisoning attacks because the server must aggregate client updates without inspecting client data. Existing robust aggregation rules often rely on update geometry, coordinate-wise robust statistics, or a single trusted reference direction, but these signals can confuse malicious behavior with benign client heterogeneity under non-i.i.d. data. We propose **CARAT**, a class-aware robust aggregation rule that uses a small labeled server reference set to sample hidden, class-balanced tasks and score each client update by its task-level loss reduction. CARAT combines these hidden-task certificates with robust clipping, common-radius evaluation, coordinate-rank anomaly penalties, and capped-simplex weight optimization to favor updates that remain useful across semantic slices while limiting structurally extreme clients. On CIFAR-100/ResNet18 with 20% malicious clients, two non-i.i.d. regimes, and four untargeted model-poisoning attacks, CARAT achieves the best average rank among the reported public baselines and improves final accuracy by 1.9–5.5 percentage points over the strongest non-CARAT baseline in each attack/heterogeneity setting. Within this scoped evaluation, the results show that task-conditioned validation is a practical alternative to purely geometric or single-anchor trust signals when a modest labeled server audit set is available.

1 Introduction

Federated learning (FL) enables many clients to train a shared model without centralizing their private data [1–3]. This distributed training paradigm is attractive for privacy- and governance-sensitive applications, but it also makes aggregation a security-critical operation. At each round, the server must update the global model from client messages whose underlying data and training process cannot be directly inspected. As a result, the aggregation rule becomes the main line of defense when some clients submit harmful updates.

Model-poisoning attacks exploit this trust bottleneck by replacing benign client updates with crafted messages. These attacks can be targeted, such as backdoor and model-replacement attacks [4–6], or untargeted, where the goal is to degrade clean test performance and destabilize collaborative training [7–9]. This paper focuses on the untargeted setting. The core challenge is not only that malicious updates can be arbitrary, but also that benign updates are naturally diverse under non-i.i.d. client data. A geometrically unusual update may therefore be either an attack or a legitimate consequence of client heterogeneity [10–13].

Existing Byzantine-robust defenses mitigate this threat through geometric selection, coordinate-wise robust statistics, or auxiliary validation, but their trust signals are often too coarse for heterogeneous FL. Krum and MultiKrum [14] select centrally located updates, Bulyan [15] combines selection with a second robust aggregation stage, coordinate-wise median [16] suppresses marginal outliers, and RFA

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

[17] uses a geometric-median objective. These rules can confuse benign dispersion with malicious deviation when client data are non-i.i.d. Auxiliary-information defenses partly address this limitation, but FLTrust [18] compresses trusted data into a single reference direction and FLDetector [19] relies on historical behavior prediction. Such signals may miss attacks whose harm is concentrated on a subset of classes or semantic regions. The resulting open problem is how to validate client updates by their task-level usefulness rather than only by their location in update space.

We propose **CARAT**, a class-aware robust aggregation rule that validates updates through hidden, class-balanced tasks sampled from a small labeled server reference set. The key idea is that a benign non-i.i.d. update need not be geometrically central, but it should provide useful evidence on semantic validation tasks that are hidden from the clients. CARAT first places candidate updates in a common scoring geometry through robust clipping and common-radius evaluation. It then measures task-level loss reduction to form hidden-task certificates, penalizes coordinate-rank anomalies, and solves a capped-simplex weight optimization problem with temporal regularization. The final update is computed from clipped client updates, so the server can preserve useful benign diversity while limiting clients that are semantically harmful or structurally extreme. This design explicitly targets settings where a modest labeled server audit set is available; it is not intended to be a fully data-free defense.

We evaluate CARAT on CIFAR-100/ResNet18 under four untargeted model-poisoning attacks, two Dirichlet non-i.i.d. regimes, and a 20% malicious-client ratio. CARAT achieves the best average rank among the reported public baselines and improves final accuracy by 1.9–5.5 percentage points over the strongest non-CARAT baseline across the tested attack/heterogeneity settings. These results support a deliberately bounded claim: class-conditioned validation can provide a practical robustness signal that outperforms purely geometric or single-anchor criteria within the completed reference-assisted evaluation scope. Our main contributions are as follows.

- We introduce hidden-task certificates for robust FL aggregation, using class-balanced server-side validation tasks to evaluate whether each client update improves semantic slices of the learning problem.
- We develop the CARAT aggregation objective, which combines hidden-task utility, coordinate-rank anomaly penalties, capped per-client influence, and temporal regularization into a single robust weighting rule.
- We provide an auditable CIFAR-100/ResNet18 evaluation against citable robust-aggregation baselines under four untargeted poisoning attacks and non-i.i.d. partitions, with matched server-reference budgets for reference-assisted defenses and seed-level reporting.

2 Related Work

Model-poisoning attacks. Model-poisoning attacks directly manipulate client model updates before aggregation; in this paper we focus on untargeted availability attacks that prevent convergence or degrade clean test performance. Early Byzantine and distributed-learning work studied large-noise attacks in Byzantine-tolerant gradient descent [14], sign-flipping attacks in asynchronous Byzantine SGD [20], and local model manipulation in federated learning [21], establishing that robust aggregation must tolerate arbitrary client messages rather than only stochastic training noise. Later attacks became substantially more adaptive. ALIE [7] exploits benign gradient variance to remain statistically plausible, IPM [22] reverses progress through inner-product manipulation, FangAttack [8] optimizes local model poisoning against specific robust aggregators, and MinMax/MinSum [9] search for malicious updates that satisfy distance- or sum-based stealth constraints while damaging the global model. Mimic-style attacks [23] further show that heterogeneity itself can be weaponized by making poisoned updates resemble legitimate client drift. Targeted backdoor and model-replacement attacks form an important adjacent threat family, including model replacement [4], distributed backdoor attacks [5], and tail-class backdoors [6], but CARAT's empirical claim is deliberately limited to untargeted model poisoning.

Defenses against model poisoning. Defenses against model poisoning differ mainly in the trust signal used for aggregation. Classical Byzantine-robust rules rely on geometry or coordinate-wise statistics: Krum and MultiKrum [14] select centrally located updates, Bulyan [15] combines selection with a second robust aggregation stage, coordinate-wise median [16] suppresses marginal outliers,

and RFA [17] uses a geometric-median objective. Subsequent defenses add suspicion scoring through Zeno [24], redundant gradients through DRACO [25], regularized aggregation through RSA [26], divide-and-conquer filtering through DnC [9], history-aware clipping through CenteredClipping [27], sign-pattern filtering through SignGuard [28], latent-space screening through FLARE [29], and layer-adaptive sparsification through LASA [30]. Because non-i.i.d. data can make benign updates geometrically dispersed, another line explicitly studies robustness under heterogeneity through FedProx [11], SCAFFOLD [12], resampling [31], bucketing [23], local-SGD analysis [32], gradient splitting [33], nearest-neighbor mixing [34], label-skew-aware aggregation [35], and entropy-guided public-data aggregation [36]; non-i.i.d. FL surveys provide the broader context [13]. Auxiliary-information defenses use trusted data, historical behavior, or diagnostic signals rather than only instantaneous update geometry: AUROR [37] screens updates in collaborative learning, FLTrust [18] bootstraps trust from a server reference update, FoolsGold [38] penalizes repeated sybil-like similarity, and FLDetector [19] predicts suspicious client behavior from history. CARAT follows this auxiliary-information direction but changes the validation principle: instead of trusting one anchor direction, one geometry statistic, or one historical detector, it asks whether each update remains useful across multiple hidden, class-balanced semantic tasks under a matched server-reference budget.

3 Preliminaries and Problem Statement

3.1 Federated Learning and Threat Model

We study synchronous FL in which the server must update a global model from client messages without inspecting client data. Let $\mathcal{N} = \{1, \ldots, N\}$ be the client population. Client i holds local data $\mathcal{D}_i$ and objective $F_i(w)$, giving the standard weighted objective

$$F(w) = \sum_{i=1}^{N} p_i F_i(w), \qquad p_i = \frac{|\mathcal{D}_i|}{\sum_{j=1}^{N} |\mathcal{D}_j|}. \tag{1}$$

At communication round r, the server broadcasts w_r to selected clients $\mathcal{S}_r$. Each selected client initializes $w_{i,r}^0 = w_r$ and performs K local stochastic-gradient steps,

$$w_{i,r}^{k+1} = w_{i,r}^k - \eta_r \nabla F_i(w_{i,r}^k; \xi_{i,r}^k), \qquad k = 0, \ldots, K-1. \tag{2}$$

The returned model message is converted to an update vector

$$\Delta_i^r = w_{i,r}^K - w_r, \tag{3}$$

and a server-side aggregation rule maps the received updates to the next global model,

$$w_{r+1} = w_r + \mathcal{A}\big(\{\Delta_i^r : i \in \mathcal{S}_r\}\big). \tag{4}$$

FedAvg is recovered when $\mathcal{A}$ is the sample-size-weighted mean.

The adversary controls a subset $\mathcal{M}_r \subseteq \mathcal{S}_r$ of selected clients and may replace their benign updates with arbitrary crafted messages $\widetilde{\Delta}_i^r$. We focus on untargeted model poisoning: malicious updates aim to degrade clean test accuracy while remaining plausible enough to evade robust aggregation. This setting is difficult under non-i.i.d. data because benign clients can naturally differ in update direction, norm, sparsity, and class emphasis. A robust aggregator must therefore reduce harmful influence without treating every geometrically atypical update as malicious client behavior.

The adversary is assumed to know the training protocol, defense family, and public hyperparameters, but not the realized hidden validation tasks sampled by the server in a given round. This is the standard security boundary for CARAT's reference-assisted setting: the labeled reference set and task sampler are server-side audit resources, not public client inputs. Attackers that directly observe the reference examples or optimize against the exact hidden tasks are outside the empirical claim set, although the method still includes clipping, rank penalties, and capped weights to reduce overreliance on the reference signal.

3.2 Reference-Assisted Robust Aggregation

CARAT assumes that the server maintains a small labeled reference set $\mathcal{D}^{\mathrm{ref}}$ covering the task label space. From this data it constructs a probe pool $\mathcal{P}$ and samples hidden, class-balanced validation

tasks during aggregation. This is a stronger setting than fully data-free robust aggregation and should be treated as part of the threat model. The distinction from single-anchor trusted-data methods is that CARAT does not compress $\mathcal{D}^{\text{ref}}$ into one trusted update direction; instead, it uses the reference data to instantiate multiple semantic checks that are hidden from clients.

Given the round-level update set $\{\Delta_i^r\}_{i \in \mathcal{S}_r}$ and reference data $\mathcal{D}^{\text{ref}}$, the server must construct nonnegative aggregation weights and an aggregate update that preserve useful benign diversity while suppressing poisoned clients. This goal imposes four design requirements. First, update scoring should be scale-controlled so that large-norm messages cannot win merely by magnitude. Second, trust should depend on task-conditioned utility, not only on distance to other clients. Third, the method should still penalize structurally extreme updates, because a harmful client may look useful on part of the probe signal. Finally, the aggregation weights should have bounded per-client influence and temporal stability across rounds. CARAT implements these requirements through common-geometry scoring, hidden-task certificates, coordinate-rank penalties, and capped-simplex optimization.

4 CARAT

4.1 Overview

CARAT turns the requirements above into a server-side aggregation procedure with three components. Figure 1 shows the pipeline. First, CARAT places candidate updates in a common scoring geometry by robustly clipping their norms and normalizing only the evaluation copies. Second, it scores each candidate by hidden-task loss reduction and augments this semantic signal with a coordinate-rank anomaly penalty. Third, it solves a capped-simplex optimization problem that favors worst-task utility while limiting structurally extreme and temporally unstable weights. The final server update is computed from clipped updates, not from the normalized evaluation copies.

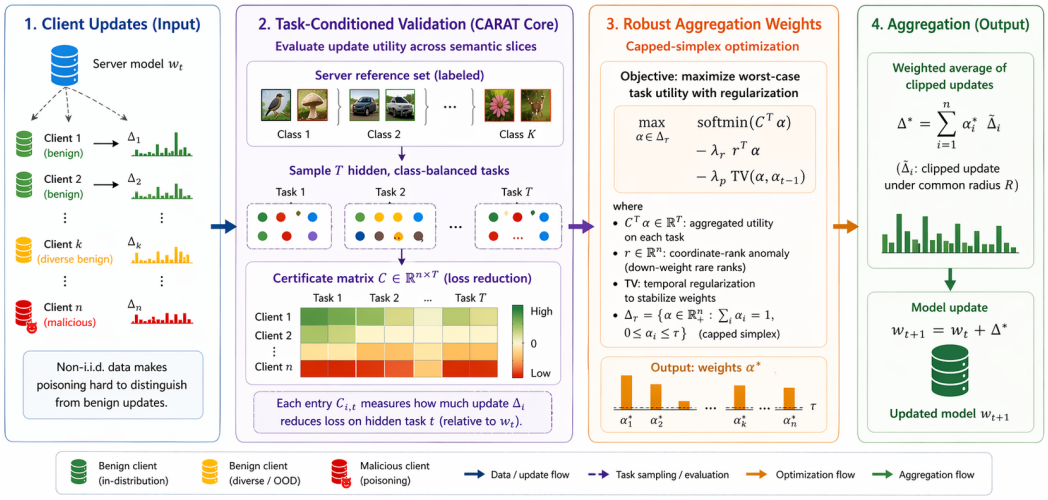

Figure 1: Overview of CARAT. Non-i.i.d. client data can make diverse benign updates difficult to distinguish from poisoning updates. CARAT uses a small labeled server reference set to sample hidden class-balanced tasks, forms a certificate matrix from task-level loss reductions, learns capped-simplex aggregation weights with rank-anomaly and temporal-stability penalties, and updates the global model by a weighted average of clipped client updates.

Table 1 summarizes why each CARAT component is included. The design is not intended as an arbitrary stack of heuristics: each term targets a distinct failure mode that appears when heterogeneous benign drift and malicious manipulation coexist.

Table 1: Design rationale for the CARAT aggregation pipeline. Each component addresses a different ambiguity in non-i.i.d. robust aggregation.

Component	Failure mode addressed	Role in CARAT
MAD clipping and common-radius scoring	Large-norm or scale-manipulated updates can dominate validation scores, while benign non-i.i.d. clients may naturally have different update norms.	Separates the scoring copy from the final clipped update, making hidden-task evaluation less sensitive to update magnitude.
Hidden class-balanced tasks	A single global score or trusted direction can miss class-local degradation and can confuse semantic usefulness with geometric centrality.	Scores each candidate by loss reduction on multiple hidden semantic slices, producing a task-level certificate matrix.
Robust task standardization	Hidden tasks can have different baseline loss scales and difficulty levels.	Makes certificate columns comparable while preserving within-task ordering.
Coordinate-rank penalty	A client may look useful on part of the reference signal while remaining structurally extreme in update space.	Penalizes consistently outlying coordinate ranks without rewarding unusual centrality.
Capped-simplex weights	Even a high-scoring client should not receive unbounded influence in a Byzantine setting.	Limits per-client weight according to an attacker-fraction prior and bounds direct malicious mass.
Temporal regularization	Round-to-round weight oscillations can overreact to noisy probe evaluations.	Encourages stable trust allocation unless hidden-task evidence supports a change.

4.2 Hidden-Task Certificates

Within a round, CARAT first constructs a scale-controlled copy of each client update for scoring. We suppress the round superscript in this section. Before semantic evaluation, the server clips each update so that a small number of large-magnitude messages cannot dominate the comparison stage. Let $\|\Delta_i\|_2$ denote the norm of client i's update. In our implementation, the clipping threshold τ_r is estimated by the MAD-based rule

$$\tau_r = \mathrm{median}(\{\|\Delta_i\|_2\}) + k \cdot 1.4826 \cdot \mathrm{MAD}(\{\|\Delta_i\|_2\}), \tag{5}$$

and the clipped update is

$$\bar{\Delta}_i = \min\left(1, \frac{\tau_r}{\|\Delta_i\|_2 + \varepsilon}\right) \Delta_i. \tag{6}$$

CARAT then rescales each clipped update to a common evaluation radius q_r,

$$\tilde{\Delta}_i = q_r \frac{\bar{\Delta}_i}{\|\bar{\Delta}_i\|_2 + \varepsilon}, \tag{7}$$

where q_r is chosen as a quantile of the positive clipped norms. The normalized vector $\tilde{\Delta}_i$ is used only to evaluate candidate utility; the final aggregation step uses $\bar{\Delta}_i$. This separation makes certificate scoring scale-neutral without changing the conservative clipped update applied to the global model.

CARAT evaluates each normalized candidate on multiple hidden semantic slices of the reference data. It samples T hidden tasks from the probe pool $\mathcal{P}$, with each task constructed to be approximately class-balanced. This class-balanced construction is the source of CARAT's class awareness: a malicious update may remain globally plausible while degrading only a subset of classes, whereas repeated balanced tasks make localized failures harder to hide. For task t, let $\ell_t(w)$ denote the average cross-entropy loss of model w on that task. CARAT defines a *hidden-task certificate*

$$c_{i,t} = \ell_t(w_r) - \ell_t(w_r + \tilde{\Delta}_i). \tag{8}$$

Here, "certificate" denotes an internal task-level utility score, not a formal certified-robustness guarantee. Positive values indicate that client i improves task t relative to the current global model.

Stacking these scores yields a certificate matrix $C \in \mathbb{R}^{n \times T}$. Because tasks may have different intrinsic difficulty and baseline loss scale, CARAT robustly standardizes each column of C before optimization. This preserves within-task ordering while making scores comparable across hidden

slices. The matrix gives the server a richer signal than client-to-client similarity alone: a client should be trusted because it remains broadly useful across hidden tasks, not merely because it lies near the geometric center of the client population.

Task utility alone is insufficient, because a malicious client may overfit the probe tasks or align with part of the semantic signal while remaining structurally extreme in parameter space. CARAT therefore augments hidden-task certificates with a coordinate-rank anomaly score. For a sampled coordinate set $S \subseteq \{1, \ldots, d\}$, it ranks clients coordinate-wise and measures the average normalized distance from the center rank:

$$\gamma_i^{(S)} = \frac{1}{|S|} \sum_{j \in S} \left| \frac{\mathrm{rank}_{i,j}}{n-1} - \frac{1}{2} \right|. \tag{9}$$

CARAT repeats this computation over several random coordinate subsets and aggregates the resulting values robustly. After robust standardization, only positive deviations are retained as the rank-penalty vector $\gamma \in \mathbb{R}^n$. This signal is intentionally asymmetric: suspicious extremeness is penalized, whereas unusual centrality is not explicitly rewarded. Together, the certificate matrix and rank penalty separate two notions that are often conflated in robust aggregation: whether an update is useful and whether it is structurally atypical.

4.3 Robust Weight Optimization

CARAT maps these signals to aggregation weights by solving a constrained optimization problem on the capped simplex

$$\mathcal{W}(\hat{\rho}) = \left\{ \alpha \in \mathbb{R}^n : \alpha_i \geq 0,\ \sum_{i=1}^{n} \alpha_i = 1,\ \alpha_i \leq \frac{1}{(1-\hat{\rho})n} \right\}, \tag{10}$$

where $\hat{\rho}$ is an estimate of the attacker fraction. The cap prevents any single client from dominating the aggregate and encodes a conservative trust budget at the weight level: even when a client appears useful, it cannot absorb arbitrary influence within one round.

Let α^{prev} be the previous round's weights. CARAT uses the quadratic temporal-variation penalty $\|\alpha - \alpha^{\mathrm{prev}}\|_2^2$ and solves

$$\max_{\alpha \in \mathcal{W}(\hat{\rho})} \ \operatorname*{softmin}_{\beta}(C^\top \alpha) - \lambda_r \gamma^\top \alpha - \lambda_p \|\alpha - \alpha^{\mathrm{prev}}\|_2^2, \tag{11}$$

where

$$\operatorname*{softmin}_{\beta}(s_1, \ldots, s_T) = -\frac{1}{\beta} \log \left(\frac{1}{T} \sum_{t=1}^{T} e^{-\beta s_t} \right). \tag{12}$$

The first term is a smooth approximation to worst-task utility: large β places more emphasis on the weakest hidden tasks, while finite β preserves optimization stability. The second term penalizes structurally extreme clients, and the third term regularizes the solution toward the previous round to reduce unnecessary oscillation. Eq. (11) implements a simple decision rule: a client should retain influence only if it remains useful across hidden tasks and does not repeatedly appear as an outlier in rank space.

After optimization, CARAT aggregates the clipped updates:

$$\Delta_{\mathrm{agg}} = \sum_{i=1}^{n} \alpha_i \bar{\Delta}_i. \tag{13}$$

This is the final server step. The aggregate inherits the robustness benefits of clipping while using weights learned from the richer evaluation geometry defined above. In practice, the main additional computation comes from evaluating n candidate updates over T hidden tasks and computing rank penalties on coordinate subsamples. Probe-model forward passes dominate, while the ranking and optimization stages remain tunable through explicit budget parameters.

A compact round-level algorithm summary is provided in Appendix A.2. Structural properties of feasible weighting, clipped aggregation, malicious-weight influence, hidden-task evaluation geometry, objective behavior, and per-round complexity are deferred to Appendix A.4.

5 Experiments

The experiments evaluate one deliberately scoped claim: whether hidden-task certificates improve robustness to untargeted model poisoning under heterogeneous client data, when compared against public robust-aggregation baselines under a fixed paper protocol and an explicit result-source policy. The main study uses CIFAR-100/ResNet18, matched trusted-data budgets for CARAT and FLTrust, final attacked accuracy/loss metrics, and average rank across attacks. Appendix B provides protocol details, final-loss results, runtime diagnostics, and reproducibility details.

5.1 Evaluation Protocol

Benchmark and threat model. We use CIFAR-100 with ResNet18 [39]. CIFAR-100 has a class-rich label space, making it suitable for evaluating whether hidden class-balanced tasks reveal class-selective degradation that may be hidden in aggregate accuracy alone. The attacked comparison covers four untargeted model-poisoning attacks: ALIE [7], FangAttack [8], MinMax [9], and MinSum [9]. These attacks test stealthy variance-based poisoning, geometry-aware local model poisoning, and distance-constrained optimization attacks. Backdoor and data-poisoning attacks remain outside the empirical claim set.

Baselines and fairness controls. The main-text comparison includes Mean, NormClipping, MultiKrum [14], FLTrust [18], FLDetector [19], and CARAT. This set spans averaging, clipping, geometry-based filtering, trusted-data anchoring, historical detection, and hidden-task validation. We exclude unpublished or under-review defenses from the main claim so that every reported baseline is citable and publicly attributable. To isolate the effect of the aggregation rule, CARAT and FLTrust use matched labeled server-reference budgets.

Training and reporting protocol. All main attacked runs use 20 clients with full participation, 200 global rounds, 4 local epochs, learning rate 0.05, and a 20% malicious-client ratio. We evaluate Dirichlet non-i.i.d. regimes with $\alpha \in \{1, 0.5\}$, where smaller α increases client heterogeneity. Clean no-attack runs are kept separate from attacked robustness tables so that clean utility is not mixed with poisoning resistance. Compatible legacy public-baseline rows use five seeds where available; regenerated FLTrust and CARAT rows use three seeds under the matched reference-assisted protocol. We disclose this source policy in the tables and use average rank to avoid overinterpreting any single seed count. Final test accuracy is the primary metric, final test loss in the appendix is a consistency check, and average rank within each non-i.i.d. setting summarizes performance across attacks.

5.2 Main Robustness Results

Table 3 reports the main attacked-accuracy comparison. The table uses the reported public baselines described above, reuses protocol-compatible five-seed legacy logs for Mean, NormClipping, MultiKrum, and FLDetector, and regenerates matched-budget FLTrust and the final $T = 8$ CARAT protocol with three seeds. The no-attack Mean run is not inserted as an attack-row baseline; it is shown only as a reference curve in Figure 2.

Within this reported comparison, CARAT obtains average rank 1.0 in both heterogeneity settings and is the strongest reported defense in every attack column. Relative to the best non-CARAT row in each column, CARAT improves mean final accuracy by 1.9–5.5 percentage points across the eight attack/heterogeneity cells. The largest margins occur under FangAttack and under the more heterogeneous $\alpha = 0.5$ MinMax/MinSum settings, where several non-CARAT defenses lose substantial accuracy. Appendix B reports the corresponding final-loss table.

Figure 2 shows the round-wise behavior under FangAttack, the attack where several non-CARAT defenses degrade most visibly in Table 3.

Table 2: Reported evaluation protocol used for the main CIFAR-100/ResNet18 robustness comparison. The table separates training controls, poisoning controls, and reporting controls.

Axis	Setting	Purpose in the comparison
Benchmark and federated training protocol		
Dataset / model	CIFAR-100 / ResNet18	Class-rich image benchmark where hidden class-balanced tasks are meaningful.
Clients	20 clients, full participation	Removes client-sampling noise from defense comparisons.
Training schedule	200 rounds; 4 local epochs; learning rate 0.05	Shared optimizer budget for all attacked runs.
Heterogeneity	Dirichlet non-i.i.d. $\alpha \in \{1, 0.5\}$	Tests robustness as benign client updates become more dispersed.
Poisoning setting and compared defenses		
Malicious clients	20% of clients	Main untargeted model-poisoning threat level.
Attacks	ALIE, FangAttack, MinMax, MinSum	Stealthy and geometry-aware untargeted poisoning attacks.
Main-text defenses	Mean, NormClipping, MultiKrum, FLTrust, FLDetector, CARAT	Public citable baselines spanning averaging, clipping, geometric filtering, trusted anchoring, and historical detection.
Reporting controls		
Metrics	Final test accuracy; final test loss; average rank	Accuracy is the main robustness metric; loss and rank provide consistency checks.
Trusted-data control	Matched labeled reference budget for FLTrust and CARAT	Prevents the proposed method from receiving more server-side supervision.
Seed policy	Legacy public-baseline rows: five seeds where available; regenerated FLTrust/CARAT rows: three seeds	Makes the result-source policy explicit while keeping the trusted-data budget matched for reference-assisted defenses.

Table 3: Final test accuracy (%) for the reported public baselines under four model-poisoning attacks on CIFAR-100/ResNet18 with 20% malicious clients and non-i.i.d. client partitions. Higher accuracy is better, and lower average rank is better. Legacy public-baseline rows reuse five-seed logs where available; FLTrust and CARAT are regenerated under the reference-assisted protocol with three seeds and matched reference budgets.

Dataset Model	Non-IID setting	Defense rule	Attacks				Avg. rank
			ALIE	**FangAttack**	**MinMax**	**MinSum**	
CIFAR-100 ResNet18	$\alpha = 1.0$	Mean	49.43 ± 0.39	48.86 ± 0.70	50.49 ± 0.59	50.50 ± 0.49	2.2
		NormClipping	25.27 ± 0.70	22.67 ± 0.77	24.86 ± 0.65	24.74 ± 0.63	6.0
		MultiKrum	39.59 ± 0.74	37.18 ± 0.83	48.69 ± 0.84	47.71 ± 1.12	4.0
		FLTrust (matched)	39.17 ± 1.87	37.68 ± 1.92	41.90 ± 1.18	41.56 ± 1.61	4.5
		FLDetector	47.96 ± 0.58	28.54 ± 4.42	50.58 ± 0.76	50.47 ± 0.27	3.2
		CARAT ($T = 8$)	51.33 ± 1.75	54.31 ± 1.28	54.35 ± 1.46	54.70 ± 1.43	1.0
CIFAR-100 ResNet18	$\alpha = 0.5$	Mean	47.62 ± 0.73	47.53 ± 1.01	48.88 ± 0.32	48.88 ± 0.52	2.2
		NormClipping	23.73 ± 0.53	20.64 ± 1.41	23.28 ± 0.40	22.92 ± 0.50	6.0
		MultiKrum	37.00 ± 0.81	33.05 ± 1.20	46.59 ± 1.09	46.45 ± 0.80	4.0
		FLTrust (matched)	34.52 ± 0.84	34.22 ± 0.94	39.50 ± 0.27	39.49 ± 0.37	4.5
		FLDetector	45.30 ± 0.75	27.77 ± 1.25	48.85 ± 0.53	49.01 ± 0.69	3.2
		CARAT ($T = 8$)	50.15 ± 1.13	51.81 ± 0.79	53.64 ± 1.12	54.10 ± 0.30	1.0

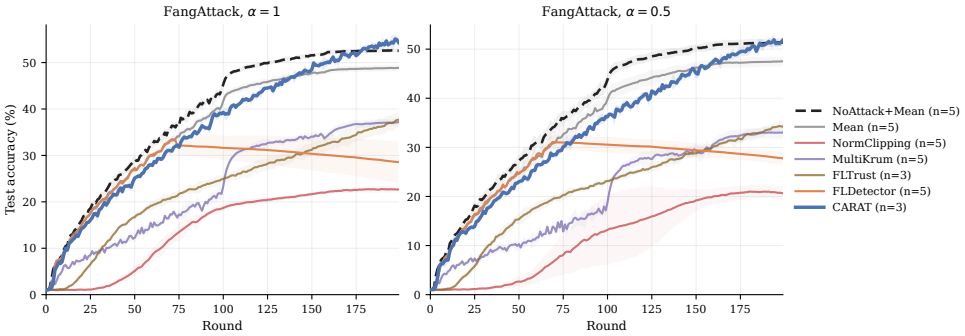

Figure 2: Evolution of test accuracy on CIFAR-100/ResNet18 under FangAttack with 20% malicious clients and non-i.i.d. data. Curves show seed means and shaded bands show one standard deviation. The dashed black curve is the no-attack Mean reference under the same client partition protocol.

5.3 Scope and Diagnostics

The completed evaluation supports a bounded claim: robustness against untargeted model poisoning on CIFAR-100/ResNet18 with 20% malicious clients and Dirichlet non-i.i.d. partitions with $\alpha \in \{1, 0.5\}$. More severe heterogeneity, larger client populations, additional datasets, backdoor attacks, data-poisoning attacks, and attackers that observe the exact hidden validation tasks remain outside the empirical claim set. This boundary is intentional: the paper evaluates whether class-aware hidden-task validation is a useful reference-assisted aggregation signal, not whether CARAT solves all Byzantine FL settings. Since Proposition A.9 identifies probe-model evaluation as CARAT's main extra cost, Appendix B.5 reports verification-scale learning curves and runtime measurements as overhead diagnostics rather than as primary robustness claims.

6 Conclusion

We presented CARAT, a class-aware robust aggregation rule that validates client updates through hidden, class-balanced task certificates rather than relying only on geometric centrality or a single trusted reference direction. Across the completed CIFAR-100/ResNet18 non-i.i.d. evaluation with matched trusted-data budgets for reference-assisted defenses, CARAT achieves the strongest performance against the reported public baselines under four untargeted model-poisoning attacks. These results support CARAT as a practical defense for FL settings where a modest labeled server audit set is available and class-selective failures matter under heterogeneous client drift.

References

[1] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial Intelligence and Statistics*, 2017.

[2] Kallista A. Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In *ACM SIGSAC Conference on Computer and Communications Security*, pages 1175–1191, 2017.

[3] Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurelien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. *Advances and Open Problems in Federated Learning*. Now Publishers, 2021.

[4] Eugene Bagdasaryan, Andreas Veit, Yiqing Hua, Deborah Estrin, and Vitaly Shmatikov. How to backdoor federated learning. In *International Conference on Artificial Intelligence and Statistics*, pages 2938–2948, 2020.

[5] Chulin Xie, Keli Huang, Pin-Yu Chen, and Bo Li. DBA: Distributed backdoor attacks against federated learning. In *International Conference on Learning Representations*, 2020.

[6] Hongyi Wang, Kartik Sreenivasan, Shashank Rajput, Harit Vishwakarma, Saurabh Agarwal, Jy-yong Sohn, Kangwook Lee, and Dimitris Papailiopoulos. Attack of the tails: Yes, you really can backdoor federated learning. In *Advances in Neural Information Processing Systems*, 2020.

[7] Gilad Baruch, Moran Baruch, and Yoav Goldberg. A little is enough: Circumventing defenses for distributed learning. In *Advances in Neural Information Processing Systems*, 2019.

[8] Minghong Fang, Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. Local model poisoning attacks to Byzantine-robust federated learning. In *USENIX Security Symposium*, 2020.

[9] Virat Shejwalkar and Amir Houmansadr. Manipulating the Byzantine: Optimizing model poisoning attacks and defenses for federated learning. In *Network and Distributed System Security Symposium*, 2021.

[10] Yue Zhao, Meng Li, Liangzhen Lai, Naveen Suda, et al. Federated learning with non-IID data. *arXiv preprint arXiv:1806.00582*, 2018.

[11] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. In *Proceedings of Machine Learning and Systems*, 2020.

[12] Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank Reddi, Sebastian Stich, and Ananda Theertha Suresh. SCAFFOLD: Stochastic controlled averaging for federated learning. In *International Conference on Machine Learning*, pages 5132–5143, 2020.

[13] Hangyu Zhu, Jinjin Xu, Shiqing Liu, and Yaochu Jin. Federated learning on non-IID data: A survey. *Neurocomputing*, 465:371–390, 2021.

[14] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. In *Advances in Neural Information Processing Systems*, 2017.

[15] El Mahdi El Mhamdi, Rachid Guerraoui, and Sébastien Rouault. The hidden vulnerability of distributed learning in Byzantium. In *International Conference on Machine Learning*, 2018.

[16] Dong Yin, Yudong Chen, Ramchandran Kannan, and Peter Bartlett. Byzantine-robust distributed learning: Towards optimal statistical rates. In *International Conference on Machine Learning*, 2018.

[17] Krishna Pillutla, Sham M. Kakade, and Zaid Harchaoui. Robust aggregation for federated learning. *IEEE Transactions on Signal Processing*, 70:1142–1154, 2022.

[18] Xiaoyu Cao, Minghong Fang, Jia Liu, Neil Zhenqiang Gong, and Yixin Gong. FLTrust: Byzantine-robust federated learning via trust bootstrapping. In *Network and Distributed System Security Symposium*, 2021.

[19] Zaixi Zhang, Xiaoyu Xu, Jinyuan Jia, Matthew Jagielski, Gauri Joshi, Mingyi Hong, and Carl A. Gunter. FLDetector: Defending federated learning against model poisoning attacks via detecting malicious clients. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022.

[20] Georgios Damaskinos, El Mahdi El Mhamdi, Rachid Guerraoui, Rhicheek Patra, and Mahsa Taziki. Asynchronous Byzantine machine learning (the case of SGD). In *International Conference on Machine Learning*, pages 1145–1154, 2018.

[21] Arjun Nitin Bhagoji, Supriyo Chakraborty, Prateek Mittal, and Seraphin Calo. Analyzing federated learning through an adversarial lens. In *International Conference on Machine Learning*, pages 634–643, 2019.

[22] Cong Xie, Oluwasanmi Koyejo, and Indranil Gupta. Fall of empires: Breaking Byzantine-tolerant SGD by inner product manipulation. In *Conference on Uncertainty in Artificial Intelligence*, pages 261–270, 2020.

[23] Sai Praneeth Karimireddy, Lie He, and Martin Jaggi. Byzantine-robust learning on heterogeneous datasets via bucketing. In *International Conference on Learning Representations*, 2022.

[24] Cong Xie, Sanmi Koyejo, and Indranil Gupta. Zeno: Distributed stochastic gradient descent with suspicion-based fault-tolerance. In *International Conference on Machine Learning*, pages 6893–6901, 2019.

[25] Lingjiao Chen, Hongyi Wang, Zachary Charles, and Dimitris Papailiopoulos. DRACO: Byzantine-resilient distributed training via redundant gradients. In *International Conference on Machine Learning*, pages 903–912, 2018.

[26] Liping Li, Wei Xu, Tianyi Chen, Georgios B. Giannakis, and Qing Ling. RSA: Byzantine-robust stochastic aggregation methods for distributed learning from heterogeneous datasets. In *AAAI Conference on Artificial Intelligence*, pages 1544–1551, 2019.

[27] Sai Praneeth Karimireddy, Lie He, and Martin Jaggi. Learning from history for Byzantine robust optimization. In *International Conference on Machine Learning*, pages 5311–5319, 2021.

[28] Jian Xu, Shao-Lun Huang, Linqi Song, and Tian Lan. SignGuard: Byzantine-robust federated learning through collaborative malicious gradient filtering. In *IEEE International Conference on Distributed Computing Systems*, pages 1026–1036, 2022.

[29] Ning Wang, Yang Xiao, Yimin Chen, Yang Hu, Wenjing Lou, and Y. Thomas Hou. FLARE: Defending federated learning against model poisoning attacks via latent space representations. In *ACM Asia Conference on Computer and Communications Security*, pages 946–958, 2022.

[30] Jiahao Xu, Zikai Zhang, and Rui Hu. Achieving Byzantine-resilient federated learning via layer-adaptive sparsified model aggregation. In *IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 1508–1517, 2025.

[31] Lie He, Sai Praneeth Karimireddy, and Martin Jaggi. Byzantine-robust learning on heterogeneous datasets via resampling. In *International Conference on Learning Representations*, 2021.

[32] Deepesh Data and Suhas Diggavi. Byzantine-resilient high-dimensional SGD with local iterations on heterogeneous data. In *International Conference on Machine Learning*, pages 2478–2488, 2021.

[33] Yuchen Liu, Chen Chen, Lingjuan Lyu, Fangzhao Wu, Sai Wu, and Gang Chen. Byzantine-robust learning on heterogeneous data via gradient splitting. In *International Conference on Machine Learning*, pages 21404–21425, 2023.

[34] Youssef Allouah, Sadegh Farhadkhani, Rachid Guerraoui, Nirupam Gupta, Rafael Pinot, and John Stephan. Fixing by mixing: A recipe for optimal byzantine ML under heterogeneity. In *International Conference on Artificial Intelligence and Statistics*, pages 1232–1300, 2023.

[35] Wenxuan Bao, Jun Wu, and Jingrui He. BOBA: Byzantine-robust federated learning with label skewness. In *International Conference on Artificial Intelligence and Statistics*, pages 892–900, 2024.

[36] Wenke Huang, Zekun Shi, Mang Ye, He Li, and Bo Du. Self-driven entropy aggregation for byzantine-robust heterogeneous federated learning. In *International Conference on Machine Learning*, pages 20096–20110, 2024.

[37] Shiqi Shen, Shruti Tople, and Prateek Saxena. AUROR: Defending against poisoning attacks in collaborative deep learning systems. In *Annual Computer Security Applications Conference*, 2016.

[38] Clement Fung, Chris J. M. Yoon, and Ivan Beschastnikh. The limitations of federated learning in sybil settings. In *International Symposium on Research in Attacks, Intrusions and Defenses*, 2020.

[39] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

Appendix organization. The appendix is organized to make each claim auditable. Appendix A gives the round-level algorithm, implementation invariants, and proofs of the structural properties used in the main text. Appendix B gives only empirical support for the completed claim: protocol details, CARAT hyperparameters, final-loss results, runtime diagnostics, and reproducibility information.

A Algorithmic and Theoretical Details

This appendix provides the technical material needed to reproduce and interpret CARAT. It first fixes the round-level notation and server inputs, then gives executable pseudocode, implementation invariants, and structural propositions for the scoring and weighting rule. These results are deliberately stated as invariants and bounds for the proposed aggregation procedure, not as a complete Byzantine-robustness guarantee against arbitrary adaptive attackers.

A.1 Notation and Round Inputs

All quantities in this appendix refer to one communication round r unless stated otherwise. The server receives $n = |\mathcal{S}_r|$ client messages and converts them into vector updates Δ_i^r. CARAT keeps three versions of each update:

$$\Delta_i^r \quad \text{(raw client update)}, \qquad \bar{\Delta}_i^r \quad \text{(clipped update)}, \qquad \tilde{\Delta}_i^r \quad \text{(common-radius scoring copy)}.$$

Only $\bar{\Delta}_i^r$ is used in the final server update. The normalized copy $\tilde{\Delta}_i^r$ is used only to evaluate hidden-task utility, so the validation score is less sensitive to residual norm differences after clipping.

The server maintains a labeled reference set $\mathcal{D}^{\mathrm{ref}}$ and derives from it a working probe pool $\mathcal{P}$. Each hidden task is a small approximately class-balanced subset sampled from $\mathcal{P}$. The raw certificate matrix is $C^{\mathrm{raw}} \in \mathbb{R}^{n\times T}$, where T is the number of hidden tasks and $C_{i,t}^{\mathrm{raw}} = c_{i,t}$ is the loss reduction obtained by candidate update i on hidden task t. After robust column-wise standardization, the optimization uses $C \in \mathbb{R}^{n\times T}$. The vector $\gamma \in \mathbb{R}^n_{\geq 0}$ denotes coordinate-rank anomaly penalties. The vector α^{prev} is the previous round's aggregation weight vector and acts as a temporal prior.

A.2 CARAT Round Algorithm

Algorithm 1 summarizes one CARAT server aggregation round. The algorithm is server-side: clients do not observe the realized hidden tasks sampled from $\mathcal{P}$.

A.3 Implementation Invariants

This subsection records implementation choices that affect reproducibility but are too low-level for the main method section. The purpose is to make the server-side scoring rule auditable, not to introduce additional assumptions beyond those stated in the threat model.

Persistent state and initialization. CARAT maintains two state variables across rounds: the probe pool $\mathcal{P}$ derived from $\mathcal{D}^{\mathrm{ref}}$, and the previous-round weight vector α^{prev}. At the first round, α^{prev} is initialized to the uniform vector. This makes the first aggregation unbiased across clients while allowing later rounds to use temporal consistency once weight trajectories have been observed.

Reference-pool construction and hidden-task sampling. Let $\mathcal{P}_c \subseteq \mathcal{P}$ denote the subset of probe examples with class label c. A hidden task is constructed by sampling a small number of examples from each available class pool and concatenating them into one balanced micro-task. If the desired number of samples per class is not available without replacement, CARAT falls back to replacement and, if needed, to a random subset of the probe pool rather than aborting the round. This fail-soft design preserves the intended semantic diversity of the task sampler while ensuring that the server always obtains a well-defined certificate matrix.

Robust score calibration. The raw certificate matrix is transformed column-wise before optimization:

$$C_{i,t} = \frac{C_{i,t}^{\mathrm{raw}} - \mathrm{median}_j(C_{j,t}^{\mathrm{raw}})}{1.4826 \cdot \mathrm{MAD}_j(C_{j,t}^{\mathrm{raw}}) + \varepsilon}.$$

Algorithm 1 CARAT server aggregation at round r

Require: Current global model w_r, client messages $\{u_i^r\}_{i=1}^n$, previous-round weights α^{prev}, server reference set $\mathcal{D}^{\text{ref}}$, probe pool $\mathcal{P}$, CARAT hyperparameters
Ensure: Aggregated update Δ_{agg}, updated temporal prior α^{prev}

```
if P is uninitialized then
    Construct a class-balanced probe pool P from D^ref
end if
if r = 1 then
    α^prev ← (1/n) 1
end if
for i = 1 to n do
    Convert u_i^r into a vector-form delta Δ_i^r
end for
Compute the clipping threshold τ_r from {‖Δ_i^r‖_2}_{i=1}^n
for i = 1 to n do
    Δ̄_i^r ← min(1, τ_r / (‖Δ_i^r‖_2 + ε)) Δ_i^r
end for
Choose the evaluation radius q_r from a quantile of positive clipped norms
for i = 1 to n do
    Δ̃_i^r ← q_r Δ̄_i^r / (‖Δ̄_i^r‖_2 + ε)
end for
Sample T hidden tasks from P using class-balanced sampling
for t = 1 to T do
    Evaluate the baseline loss ℓ_t(w_r)
end for
for i = 1 to n do
    for t = 1 to T do
        c_{i,t} ← ℓ_t(w_r) − ℓ_t(w_r + Δ̃_i^r)
    end for
end for
Robustly standardize certificate columns to obtain C
Compute rank penalties γ from repeated coordinate subsamples of {Δ̄_i^r}_{i=1}^n
α^(0) ← α^prev
for k = 0 to K − 1 do
    z ← C^⊤ α^(k)
    π ← softmin task weights induced by z and β
    g ← Cπ − λ_r γ − 2λ_p (α^(k) − α^prev)
    α^(k+1) ← Π_{W(ρ̂)} (α^(k) + ηg)
end for
α* ← α^(K) and α^prev ← α*
Δ_agg ← Σ_{i=1}^n α_i* Δ̄_i^r
return Δ_agg, α^prev
```

This removes task-specific scale effects while preserving within-task ordering by Proposition A.5. Rank penalties are calibrated similarly. For each coordinate subsample S_s, let

$$\rho_i^{(s)} = \frac{1}{|S_s|} \sum_{j \in S_s} \left| \frac{\text{rank}_{i,j}}{n-1} - \frac{1}{2} \right|.$$

CARAT then computes a standardized rank score

$$z_i^{(s)} = \frac{\rho_i^{(s)} - \text{median}_\ell(\rho_\ell^{(s)})}{1.4826 \cdot \text{MAD}_\ell(\rho_\ell^{(s)}) + \varepsilon},$$

and aggregates subsamples through

$$\gamma_i = \max\left(0, \text{median}_s\, z_i^{(s)}\right).$$

Thus a client is penalized only when it is consistently extreme across repeated coordinate views, not because of a single noisy subsample.

Projected optimization. CARAT initializes the projected optimizer with $\alpha^{(0)} = \alpha^{\text{prev}}$. If

$$z(\alpha) = C^\top \alpha, \qquad \pi_t(\alpha) = \frac{\exp(-\beta z_t(\alpha))}{\sum_{\ell=1}^{T} \exp(-\beta z_\ell(\alpha))},$$

then the ascent direction used in projected updates is

$$g(\alpha) = C\pi(\alpha) - \lambda_r \gamma - 2\lambda_p(\alpha - \alpha^{\text{prev}}).$$

With optimizer step size η, one iteration takes the form

$$\alpha^{(k+1)} = \Pi_{\mathcal{W}(\hat{\rho})}\left(\alpha^{(k)} + \eta g(\alpha^{(k)})\right).$$

The projection maintains feasibility throughout optimization, so every intermediate iterate satisfies nonnegativity, unit-sum, and per-client cap constraints.

Diagnostics. The implementation records learned weights, task scores, softmin task weights, rank penalties, clipping thresholds, evaluation radii, and hidden-task identifiers. These diagnostics distinguish three qualitatively different behaviors: benign but diverse clients that retain moderate weight, harmful clients that lose certificate support across hidden tasks, and structurally extreme clients whose influence is reduced mainly through the rank penalty.

A.4 Proofs and Structural Properties

The following propositions are grouped by the role they play in CARAT. Propositions A.1–A.3 characterize feasible weighted aggregation and bounded influence after clipping. Propositions A.4–A.5 characterize the hidden-task evaluation geometry. Propositions A.6–A.8 characterize the optimization objective, and Proposition A.9 records the per-round cost.

Proposition A.1 (Feasibility and support of the capped simplex)**.** *Let*

$$\mathcal{W}(\hat{\rho}) = \left\{\alpha \in \mathbb{R}^n : \alpha_i \geq 0,\ \sum_{i=1}^{n} \alpha_i = 1,\ \alpha_i \leq \frac{1}{(1-\hat{\rho})n}\right\},$$

with $\hat{\rho} < 1$. Then $\mathcal{W}(\hat{\rho})$ is non-empty, and every $\alpha \in \mathcal{W}(\hat{\rho})$ satisfies

$$|\operatorname{supp}(\alpha)| \geq (1-\hat{\rho})n.$$

Proof. Let $u = 1/((1-\hat{\rho})n)$. Since $\hat{\rho} < 1$, we have $u \geq 1/n$, so the uniform weight vector is feasible and the set is non-empty. For the support bound, if only k coordinates of α are positive, then

$$1 = \sum_{i=1}^{n} \alpha_i \leq ku = \frac{k}{(1-\hat{\rho})n},$$

which implies $k \geq (1-\hat{\rho})n$. Therefore CARAT can suppress suspicious clients, but cannot collapse the aggregate onto an arbitrarily small subset of participants. □

Proposition A.2 (Norm bound for the final aggregate)**.** *Suppose each clipped update satisfies $\|\bar{\Delta}_i\|_2 \leq \tau_r$ and $\alpha \in \mathcal{W}(\hat{\rho})$. Then the final aggregate*

$$\Delta_{\text{agg}} = \sum_{i=1}^{n} \alpha_i \bar{\Delta}_i$$

satisfies

$$\|\Delta_{\text{agg}}\|_2 \leq \tau_r.$$

Proof. Because $\alpha_i \geq 0$ and $\sum_i \alpha_i = 1$, the aggregate is a convex combination of clipped updates. By the triangle inequality,

$$\|\Delta_{\text{agg}}\|_2 \leq \sum_{i=1}^{n} \alpha_i \|\bar{\Delta}_i\|_2 \leq \sum_{i=1}^{n} \alpha_i \tau_r = \tau_r.$$

Thus the final server step inherits the clipping radius as a hard norm upper bound. □

Proposition A.3 (Influence bound from malicious weight mass)**.** *Let $\mathcal{M}$ be the malicious clients selected in a round and let*

$$q_{\mathcal{M}} = \sum_{i \in \mathcal{M}} \alpha_i$$

be their total CARAT weight. Suppose every clipped update satisfies $\|\bar{\Delta}_i\|_2 \leq \tau_r$. Then the direct malicious contribution obeys

$$\left\| \sum_{i \in \mathcal{M}} \alpha_i \bar{\Delta}_i \right\|_2 \leq q_{\mathcal{M}} \tau_r.$$

If $q_{\mathcal{M}} < 1$ and

$$\Delta_{\mathcal{B}} = \sum_{i \notin \mathcal{M}} \frac{\alpha_i}{1 - q_{\mathcal{M}}} \bar{\Delta}_i$$

denotes the benign-only aggregate obtained by renormalizing the benign weights, then

$$\|\Delta_{\text{agg}} - \Delta_{\mathcal{B}}\|_2 \leq 2 q_{\mathcal{M}} \tau_r.$$

Proof. The first inequality follows from the triangle inequality:

$$\left\| \sum_{i \in \mathcal{M}} \alpha_i \bar{\Delta}_i \right\|_2 \leq \sum_{i \in \mathcal{M}} \alpha_i \|\bar{\Delta}_i\|_2 \leq q_{\mathcal{M}} \tau_r.$$

For the second inequality, write

$$\Delta_{\text{agg}} = (1 - q_{\mathcal{M}}) \Delta_{\mathcal{B}} + \sum_{i \in \mathcal{M}} \alpha_i \bar{\Delta}_i.$$

Therefore

$$\|\Delta_{\text{agg}} - \Delta_{\mathcal{B}}\|_2 \leq q_{\mathcal{M}} \|\Delta_{\mathcal{B}}\|_2 + \left\| \sum_{i \in \mathcal{M}} \alpha_i \bar{\Delta}_i \right\|_2.$$

Since $\Delta_{\mathcal{B}}$ is a convex combination of clipped benign updates, $\|\Delta_{\mathcal{B}}\|_2 \leq \tau_r$ by Proposition A.2. Combining this with the first inequality gives the stated bound. □

Proposition A.4 (Directional invariance of certificate scoring)**.** *Suppose two nonzero clipped updates satisfy $\bar{\Delta}_i = a \bar{\Delta}_j$ for some scalar $a > 0$. Then their normalized versions in Eq.* (7) *coincide, i.e.,*

$$\tilde{\Delta}_i = \tilde{\Delta}_j.$$

Consequently, their hidden-task certificates are identical:

$$c_{i,t} = c_{j,t} \qquad \textit{for all } t \in \{1, \ldots, T\}.$$

Proof. Eq. (7) rescales every nonzero clipped update to the common radius q_r. Positive collinearity is therefore removed by normalization:

$$q_r \frac{\bar{\Delta}_i}{\|\bar{\Delta}_i\|_2} = q_r \frac{a \bar{\Delta}_j}{\|a \bar{\Delta}_j\|_2} = q_r \frac{\bar{\Delta}_j}{\|\bar{\Delta}_j\|_2}.$$

Substituting the resulting equality into Eq. (8) gives identical task scores. Thus, after clipping, the certificate stage depends on direction rather than residual norm magnitude. □

Proposition A.5 (Task-wise standardization preserves within-task ordering)**.** *Fix a task t and define a standardized certificate score*

$$\hat{c}_{i,t} = \frac{c_{i,t} - m_t}{s_t + \varepsilon},$$

where $m_t \in \mathbb{R}$ and $s_t > 0$ depend only on task t and not on client i. Then for any clients i, j,

$$c_{i,t} \geq c_{j,t} \quad \Longleftrightarrow \quad \hat{c}_{i,t} \geq \hat{c}_{j,t}.$$

Moreover,

$$\hat{c}_{i,t} - \hat{c}_{j,t} = \frac{c_{i,t} - c_{j,t}}{s_t + \varepsilon}.$$

Proof. For a fixed task t, the map $x \mapsto (x - m_t)/(s_t + \varepsilon)$ is affine with strictly positive slope because $s_t + \varepsilon > 0$. Hence it preserves ordering. The difference identity follows by direct subtraction. Therefore column-wise robust standardization calibrates cross-task scale without changing the within-task ranking induced by the raw certificate scores. □

Proposition A.6 (Softmin bounds and gradient form)**.** *Let $s \in \mathbb{R}^T$ and define*

$$\operatorname*{softmin}_{\beta}(s) = -\frac{1}{\beta} \log \left(\frac{1}{T} \sum_{t=1}^{T} e^{-\beta s_t} \right).$$

Then

$$\min_t s_t \le \operatorname*{softmin}_{\beta}(s) \le \min_t s_t + \frac{\log T}{\beta}.$$

Moreover,

$$\lim_{\beta \to \infty} \operatorname*{softmin}_{\beta}(s) = \min_t s_t, \qquad \lim_{\beta \to 0} \operatorname*{softmin}_{\beta}(s) = \frac{1}{T} \sum_{t=1}^{T} s_t.$$

If $z(\alpha) = C^\top \alpha$ and

$$\pi_t(\alpha) = \frac{\exp(-\beta z_t(\alpha))}{\sum_{\ell=1}^{T} \exp(-\beta z_\ell(\alpha))},$$

then

$$\nabla_\alpha \operatorname*{softmin}_{\beta}(C^\top \alpha) = C\pi(\alpha).$$

Proof. Let $m = \min_t s_t$. Since

$$e^{-\beta m}/T \le \frac{1}{T} \sum_{t=1}^{T} e^{-\beta s_t} \le e^{-\beta m},$$

applying $-(1/\beta)\log(\cdot)$ yields

$$m \le \operatorname*{softmin}_{\beta}(s) \le m + \frac{\log T}{\beta}.$$

The limit $\beta \to \infty$ follows from the vanishing gap bound. For $\beta \to 0$, a first-order expansion of the exponential average gives

$$\frac{1}{T} \sum_{t=1}^{T} e^{-\beta s_t} = 1 - \beta \frac{1}{T} \sum_{t=1}^{T} s_t + o(\beta),$$

so the softmin converges to the arithmetic mean. Differentiating the softmin expression with respect to z_t yields the Gibbs weights $\pi_t(\alpha)$, and the chain rule gives $C\pi(\alpha)$. Hence the ascent direction is a convex combination of certificate columns, with more weight placed on tasks whose current aggregate utility is smaller. □

Proposition A.7 (Temporal prior and projected-iterate invariants)**.** *Define*

$$f(\alpha) = \operatorname*{softmin}_{\beta}(C^\top \alpha) - \lambda_r \gamma^\top \alpha - \lambda_p \|\alpha - \alpha^{\text{prev}}\|_2^2.$$

If $\alpha, \alpha' \in \mathcal{W}(\hat{\rho})$ satisfy $C^\top \alpha = C^\top \alpha'$ and $\gamma^\top \alpha = \gamma^\top \alpha'$, then

$$f(\alpha) - f(\alpha') = -\lambda_p \left(\|\alpha - \alpha^{\text{prev}}\|_2^2 - \|\alpha' - \alpha^{\text{prev}}\|_2^2 \right).$$

In addition, under projected gradient ascent

$$\alpha^{(k+1)} = \Pi_{\mathcal{W}(\hat{\rho})} \left(\alpha^{(k)} + \eta \nabla f(\alpha^{(k)}) \right),$$

every iterate remains in $\mathcal{W}(\hat{\rho})$.

Proof. The first statement follows by direct substitution: if the certificate and rank terms match, the only remaining difference in objective value is the quadratic prior term. CARAT therefore prefers, among utility-equivalent feasible weights, the solution closer to α^{prev}. The second statement is immediate from the projection operator. Hence the optimizer preserves three round invariants throughout: $\alpha_i \geq 0$, $\sum_i \alpha_i = 1$, and $\alpha_i \leq 1/((1-\hat{\rho})n)$. □

Proposition A.8 (Pairwise weight-transfer condition)**.** *Let*

$$f(\alpha) = \operatorname*{softmin}_{\beta}(C^\top \alpha) - \lambda_r \gamma^\top \alpha - \lambda_p \|\alpha - \alpha^{\text{prev}}\|_2^2$$

be the CARAT objective, and let e_a, e_b be coordinate vectors for two clients a and b. Suppose $\alpha + \epsilon(e_b - e_a)$ remains feasible for sufficiently small $\epsilon > 0$. Define

$$\pi_t(\alpha) = \frac{\exp(-\beta(C^\top \alpha)_t)}{\sum_{\ell=1}^{T} \exp(-\beta(C^\top \alpha)_\ell)}.$$

Then moving infinitesimal weight from client a to client b increases the objective to first order whenever

$$(C_b - C_a)^\top \pi(\alpha) - \lambda_r(\gamma_b - \gamma_a) - 2\lambda_p[(\alpha_b - \alpha_b^{\text{prev}}) - (\alpha_a - \alpha_a^{\text{prev}})] > 0,$$

where C_i denotes row i of the standardized certificate matrix.

Proof. Let $d = e_b - e_a$. A first-order expansion gives

$$f(\alpha + \epsilon d) - f(\alpha) = \epsilon d^\top \nabla f(\alpha) + o(\epsilon).$$

Using Proposition A.6, the gradient is

$$\nabla f(\alpha) = C\pi(\alpha) - \lambda_r \gamma - 2\lambda_p(\alpha - \alpha^{\text{prev}}).$$

Substituting $d = e_b - e_a$ yields exactly the displayed expression. Hence a client gains weight when its hidden-task utility advantage, measured through the current softmin focus $\pi(\alpha)$, outweighs its rank penalty and temporal-prior cost. □

Proposition A.9 (Per-round complexity)**.** *Let n be the number of clients, T the number of hidden tasks, m the number of sampled coordinates for rank scoring, S the number of rank subsamples, and K the number of projected-gradient steps. Ignoring constant factors from batch size and model width, the extra defense-side work of CARAT scales as*

$$\textit{probe-model forward passes} + O(Snm + KnT).$$

Proof. CARAT performs one baseline probe evaluation plus n candidate probe evaluations over the T hidden tasks, then computes S rank summaries over m coordinates, and finally executes K projected optimization steps over n weights and T task scores. In the prototype implementation, the probe-model forward passes dominate wall-clock time, while the rank and optimization stages are lower-order terms controlled by m, S, and K. □

Remark on the reference assumption. The object $\mathcal{D}^{\text{ref}}$ denotes the labeled server reference set, while $\mathcal{P}$ is a subsampled working pool derived from it for hidden-task construction. This distinction is part of CARAT's threat model and materially strengthens the setting relative to fully data-free robust aggregation.

B Experimental Protocols and Additional Results

This part records only empirical material that directly supports the main-text claims. It first fixes the completed CIFAR-100/ResNet18 protocol and CARAT hyperparameters, then reports final-loss results, a runtime diagnostic, and reproducibility details.

B.1 Completed Empirical Scope

The completed empirical scope is intentionally narrow. We evaluate untargeted model poisoning on CIFAR-100/ResNet18 with Dirichlet non-i.i.d. client partitions, 20% malicious clients, and four attacks: ALIE, FangAttack, MinMax, and MinSum. The comparison uses Mean, NormClipping, MultiKrum, FLTrust, FLDetector, and CARAT. All claims are based on attacked final test accuracy, attacked final test loss, and average rank. CARAT and FLTrust use matched labeled server-reference budgets, unpublished or under-review defenses are excluded from the claim set, and the result-source policy is disclosed for each table rather than hidden in aggregate statistics.

B.2 Reported CIFAR-100 Protocol

Unless stated otherwise, the CIFAR-100 comparison uses 20 clients with full participation, 200 communication rounds, 4 local epochs, learning rate 0.05, and Dirichlet non-i.i.d. partitions with $\alpha \in \{1, 0.5\}$. Legacy public-baseline rows use five random seeds where available, while regenerated FLTrust and CARAT rows use three random seeds under the matched reference-assisted protocol.

Table 4: CIFAR-100/ResNet18 protocol used for the reported public-baseline result set.

Component	**Setting**	**Role in the paper**
Dataset and model	CIFAR-100 / ResNet18	Primary class-rich benchmark for the main robustness claim.
Client population	20 clients, full participation	Keeps client sampling fixed so defense differences are not confounded with participation noise.
Training schedule	200 rounds, 4 local epochs, learning rate 0.05	Shared optimization protocol for attacked runs.
Data heterogeneity	Dirichlet non-i.i.d. $\alpha \in \{1, 0.5\}$	Completed heterogeneous settings used for the main claim.
Malicious clients	20%	Attacker fraction used in the completed result set.
Attack suite	ALIE, FangAttack, MinMax, MinSum	Untargeted model-poisoning attacks used for final tables and ranks.
Compared defenses	Mean, NormClipping, MultiKrum, FLTrust, FLDetector, CARAT	Public, citable baselines used in Table 3.
Seeds and source policy	Legacy rows: five seeds where available; regenerated FLTrust/CARAT rows: three seeds	Compatible legacy logs are reused with explicit source disclosure; reference-assisted rows are regenerated with matched trusted-data budgets.
Trusted-data budget	Matched between FLTrust and CARAT	Prevents CARAT from receiving a larger labeled server-reference budget than the trusted-data baseline.

B.3 CARAT Hyperparameters

Table 5 lists the CARAT defaults used in the reported CIFAR-100 result set. These values define the reported protocol and make the server-side evaluation budget explicit.

B.4 Final-Loss Results

Table 6 reports the final test loss corresponding to Table 3. It uses the same source policy: compatible legacy public-baseline logs are reused, while matched-budget FLTrust and $T = 8$ CARAT are taken from the regenerated three-seed protocol.

Table 5: CARAT hyperparameters used in the reported CIFAR-100/ResNet18 public-baseline runs.

Component	Default value	Purpose
Hidden-task evaluation	8 tasks, 4 probe samples per class, batch size 128	Builds repeated class-balanced certificates for candidate client updates.
Normalization and clipping	Probe-radius quantile 0.5; MAD clipping factor 2.5	Places candidates on a common evaluation radius and limits residual norm influence.
Feasible weight set	Attacker prior $\hat{\rho} = 0.2$	Caps per-client weight and prevents collapse onto a very small client subset.
Task and anomaly weighting	Certificate temperature $\beta = 12.0$; rank weight 0.20; prior weight 0.05	Balances weakest-task utility, structural rank penalty, and temporal smoothness.
Rank diagnostics	4096 sampled coordinates, 5 rank subsamples	Estimates whether a client is consistently extreme across coordinate views.
Projected optimization	60 projected steps, step size 0.5	Solves the capped-simplex weighting problem used for aggregation.

Table 6: Final test loss under four model-poisoning attacks on CIFAR-100/ResNet18 with 20% malicious clients and non-i.i.d. client partitions. Lower is better. Legacy public-baseline rows reuse five-seed logs where available; FLTrust and CARAT are regenerated under the reference-assisted protocol with three seeds and matched reference budgets.

Dirichlet α	Defense	ALIE	FangAttack	MinMax	MinSum	Avg. rank
1.0	Mean	0.0292 ± 0.0004	0.0294 ± 0.0004	0.0283 ± 0.0004	0.0283 ± 0.0006	2.2
1.0	NormClipping	0.0476 ± 0.0005	0.0501 ± 0.0004	0.0484 ± 0.0004	0.0485 ± 0.0003	6.0
1.0	MultiKrum	0.0361 ± 0.0006	0.0377 ± 0.0005	0.0295 ± 0.0005	0.0301 ± 0.0009	4.0
1.0	FLTrust (matched)	0.0363 ± 0.0016	0.0375 ± 0.0015	0.0344 ± 0.0008	0.0346 ± 0.0013	4.5
1.0	FLDetector	0.0305 ± 0.0004	0.0450 ± 0.0035	0.0283 ± 0.0006	0.0284 ± 0.0003	3.2
1.0	**CARAT** ($T = 8$)	0.0279 ± 0.0010	0.0256 ± 0.0008	0.0255 ± 0.0005	0.0252 ± 0.0009	1.0
0.5	Mean	0.0303 ± 0.0005	0.0302 ± 0.0006	0.0293 ± 0.0001	0.0293 ± 0.0004	2.2
0.5	NormClipping	0.0486 ± 0.0005	0.0517 ± 0.0011	0.0497 ± 0.0005	0.0498 ± 0.0004	6.0
0.5	MultiKrum	0.0382 ± 0.0008	0.0407 ± 0.0010	0.0306 ± 0.0006	0.0308 ± 0.0006	4.0
0.5	FLTrust (matched)	0.0396 ± 0.0005	0.0403 ± 0.0005	0.0362 ± 0.0002	0.0361 ± 0.0002	4.5
0.5	FLDetector	0.0319 ± 0.0004	0.0458 ± 0.0011	0.0294 ± 0.0004	0.0292 ± 0.0005	3.2
0.5	**CARAT** ($T = 8$)	0.0285 ± 0.0012	0.0272 ± 0.0008	0.0258 ± 0.0008	0.0258 ± 0.0004	1.0

B.5 Verification-Scale Runtime Results

Figure 3 reports a smaller CIFAR-100/i.i.d. ALIE verification run that records both round-wise accuracy and server-side runtime. In this setting, CARAT remains stable across seeds under attack but pays a clear runtime premium because each round evaluates candidate updates on repeated hidden tasks. The result is not used as the main robustness claim; it is included only to make the overhead tradeoff visible.

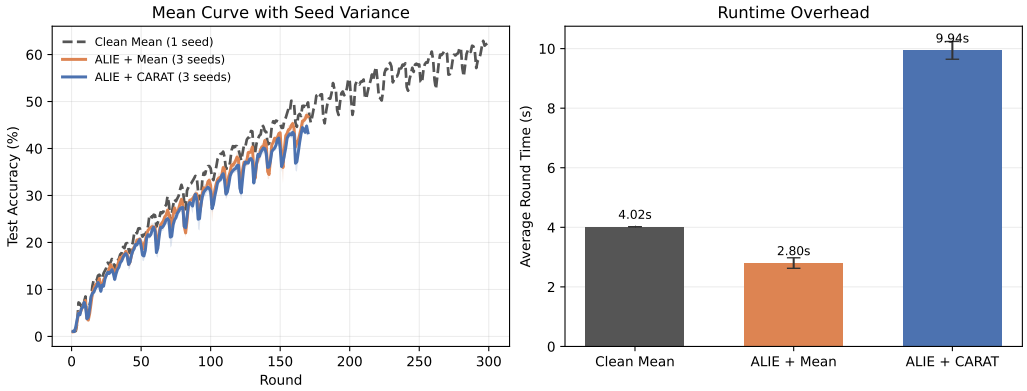

Figure 3: Verification-scale local results on CIFAR-100 under the ALIE model-poisoning attack. Left: round-wise test accuracy, where solid curves show the mean over three seeds and shaded bands denote one standard deviation; the dashed clean Mean curve is a single-seed no-attack reference. Right: average per-round wall-clock time with seed-level error bars, showing the defense overhead of repeated hidden-task evaluation.

B.6 Reproducibility, Ethics, and Availability

Reproducibility. All main reported results use the public CIFAR-100 benchmark and the fixed training protocol described in Section 5. The aggregation method is specified in Algorithm 1, with implementation defaults listed above. For anonymous review, logs, plotting scripts, and configuration files should be included in the supplementary material or an anonymized archive so that each table cell can be traced to its seed-level runs.

Compute resources. The reported public-baseline logs were generated through the repository launch scripts on a Compute Canada SLURM cluster. All runs use the same launcher, export script, and fixed configuration files. Each run writes a metadata file recording the configuration file, command line, seed, Python environment, CUDA requirement, and git commit.

Ethics. This work studies defenses against model-poisoning attacks in federated learning. The attack implementations are used to evaluate robustness and do not require access to private user data. The main practical risk is dual use: stronger attack simulations may help adversaries tune poisoning strategies. We mitigate this by framing attacks as benchmark stressors and by releasing defensive evaluation code together with the corresponding protocols.

Data availability. CIFAR-100 is a public dataset. CARAT uses a small labeled server-side reference split during evaluation; when reproducing the results, this split should be generated deterministically from the public training data under the same seed and excluded from client local training data.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes].

   Justification: The abstract and introduction state the method, assumptions, benchmark scope, and main result. The threat model, evaluation scope, and conclusion narrow the claim to untargeted model poisoning with server-side reference data.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes].

   Justification: The main text discusses limitations where they affect interpretation: Section 3 states the server-side reference-data assumption, Section 5 states the empirical and attack-family scope, and Appendix B.5 reports runtime overhead diagnostics.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes].

Justification: The formal results are stated as propositions with assumptions and proof sketches in Appendix A.4. They are per-round aggregation properties rather than a full convergence theorem.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes].

Justification: Section 5 and Appendix B specify datasets, attacks, defenses, client counts, training schedule, hyperparameters, seed counts, and reporting protocol for the main results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in

some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [No].

   Justification: The paper describes the required code, configurations, logs, and public datasets, but the final anonymized code archive must still be attached or linked for the submission.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes].

   Justification: The main text gives the training setup and Table 2 summarizes the protocol. Appendix B adds hyperparameters, stress-test axes, and ablation details.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes].

   Justification: Tables report mean and standard deviation over seeds where available, and the captions state CARAT seed counts explicitly. The one single-seed CARAT cell is identified in the main text and is not used for variance-sensitive conclusions.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes].

   Justification: Appendix B.6 reports the Compute Canada SLURM setup, GPU type, CPU and memory request, wall-time limits, array concurrency, and run metadata recorded by the launcher.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes].

   Justification: The work uses public datasets and evaluates defenses against simulated attacks. Appendix B.6 discusses ethics and dual-use considerations.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes].

Justification: Appendix B.6 discusses positive impacts for FL security and the dual-use risk of attack simulations.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A].

Justification: The paper does not release a high-risk model, scraped dataset, or generative system requiring special access safeguards.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No].

Justification: The paper cites the benchmark datasets and related methods, but dataset license details and third-party code license details should be verified before final submission.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.

- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [N/A].

    Justification: The paper introduces an algorithm and experimental implementation, but it does not introduce a new dataset, pretrained model, or benchmark asset.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A].

    Justification: The work does not involve crowdsourcing or human-subject experiments.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A].

Justification: The work does not involve crowdsourcing or human-subject experiments, so IRB approval is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A].

Justification: LLMs are not part of the core method, experiments, or scientific contribution. Any language editing assistance does not affect the methodology or reported results.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.